The IGCSE Pseudocode to Python Reference Guide

Eason Qin Luojia (eason@ezntek.com)

Eighth Revision

August 20th, 2026

What is this document?

This document provides a side-by-side comparison/reference as to the differences between IGCSE Pseudocode and Python. Please know this **does not cover and does not intend to teach HOW** to program in pseudocode! This is only helpful for Python programmers who need to learn IGCSE Pseudocode to sit exams, or the other way around if they want to write useful programs without a special interpreter like beancode (You can use the online version at https://beancode-web.pages.dev)

You may also visit https://ezntek.com/revision to find a digital copy of this document.

Note 1

All values in angle brackets, like so**:**

```
<variable name>

<type>

<value>
```

represent *meta-variables* or *meta-values*, which represents a *placeholder* for an actual value. Everything between <> should be replaced with what it *says* inside. You should not write the <> either. For example, if `<type>` is given, replace it with a type, like INTEGER.

If there is an item that leaks onto a new line, such as,

```
FOR <counter> ← <begin> TO <end> STEP
<step>
    <statement>…
NEXT <counter>
```

Count it as if it were equal to:

```
FOR <counter> ← <begin> TO <end> STEP <step>
    <statement>…
NEXT <counter>
```

Note 2

Some key definitions will be made**:**

Term	Meaning
Expression `<expr>`	Any **variable name or value, function calls, or arithmetic expressions like addition, enclosed or not enclosed in brackets**. It **will** be shortened to **expr** when necessary.
Identifier `<ident>`	A **variable name**. It **will** be shortened to **ident** when necessary.
Operator	a symbol that does something, such as math. They include symbols such as `*` `+` `-` `/` etc.

…	Represents repetition, i.e. repeated statements. If there is a comma, such as
	`<statement>, …`
	That implies that there can either be one statement <statement>, or many statements separated by a comma, such as
	`<statement>, <statement>, <statement>`

Note 3

This work is copyright © Eason Qin 2024—2026, and is licensed under the Creative Commons Attribution-ShareAlike-NonCommercial 4.0 International license. Visit the Creative Commons Website (https://creativecommons.org) for details. **All prior versions are also licensed under the same license.**

Note 4

This is the **eighth revision** of the guide. If you have earlier revisions, view the changelog:

1. **Initial version**.
2. **Fixed syntax highlighting** added consistency in the *Functions* section, and added this note.
3. **Added a License.**
4. **Fixed inconsistencies in the notes,** and slight syntax highlighting changes
5. **Fixed critical error in the For loop section**
6. **Fixed if statement indentations and array syntax**
7. **Some syntax cleanups and cleanups to notices**
8. **Clarified some explanations, rephrased certain sections and clarified Python lists.**

Reference Guide

Item	IGCSE Pseudocode	Python

Comment
Used to annotate code.
	`// This is a comment.`
	`// To comment, simply put two`
	`// slashes (//) in front of your text.`
		`# This is a comment.`
		`# To comment, simply put one`
		`# hashtag (#) in front of your`
		`# text.`

Values and Types
 	
"Values" are known as literals, they represent values.
	`// These are all INTEGER's, or whole`
	`// numbers`
	`42`
	`-2043`
	
	`// These are all REAL's, or decimal`
	`// numbers`
	`3.14159`
	`2.718282`
	`56.52`
	
	`// These are STRING's, or "text"`
	`// (enclosed in only "):`
	`"Good morning, user!"`
	`"Thomas"`
	`"Jason Lee"`
	
	`// These are BOOLEAN's, either TRUE or FALSE`
	`TRUE`
	`FALSE`
	
	`// These are CHAR's, or singular`
	`// characters (enclosed only in '):`
	`'c'`
	`'F'`
	`'b'`
		`# These are all int's, or whole`
		`# numbers`
		`42`
		`-2043`
		
		`# these are all float's, or decimal`
		`# numbers`
		`3.14159`
		`2.718282`
		`56.52`
		
		`# These are str's, or "text"`
		`# (enclosed in either " and ')`
		`"Good morning, user!"`
		`'Thomas'`
		`'Jason Lee'`
		
		`# These are bool's, either TRUE or`
		`# FALSE`
		`True`
		`False`
		
		`# there is no CHAR in Python, just use a str.`
		`"c"`
		`"F"`
		`"b"`

Declaring a variable
This is to make it clear to the computer that the variable exists.
 	
This is not necessary in Python. *You can still do it, however.*
	`DECLARE <variable name>: <type>`
	
	`// e.g.`
	`DECLARE Name: STRING`
	`DECLARE TotalScore: INTEGER`
		`<variable name>: <type>`
		
		`# e.g.`
		`name: str`
		`total_score: int`
		
		`# Types are completely optional in`
		`# Python. All occurrences of`
		`# <name>: type are for clarity.`

Assignment
This is used to give a value to a previously declared variable.
	`<variable name> ← <expression>`
	`// NOTE: you may write it like <- in`
	`// your computer.`
	
	`// e.g.`
	`Name ← "Thomas"`
	`TotalScore ← 84`
	`Name ← FirstName`
		`<variable name> = <expression>`
		
		`# e.g.`
		`name = "Thomas"`
		`total_score = 84`
		`name = first_name`

Input and Output
This is used to give users feedback and receive input.
	`OUTPUT <expression>`
	`OUTPUT <expression>, ...`
	`// Print however many things you`
	`// require.`
	
	`// There is no way to show a prompt.`
	`INPUT <expression>`
	
	`// e.g.`
	`OUTPUT "What is your name"`
	`OUTPUT "Welcome", Name`
	
	`INPUT SocialSecurityNumber`
	`OUTPUT "What is your ID?"`
	`INPUT ID`
		`print(<expression>)`
		`print(<expression>, ...)`
		`# Print however many things you`
		`# require.`
		
		`# The prompt is optional.`
		`<variable name> = input(<prompt>)`
		
		`# e.g.`
		`print("What is your name")`
		`print("Welcome", name)`
		
		`# Note that if you need to input`
		`# something into an integer, you must`
		`# wrap input in int, or separate them`
		`# like so:`
		`social_security_number = int(input())`
		`id = input("What is your ID?")`
		`id = int(id)`

Arithmetic (expression)
This is to do math.
	`<expr> <operator> <expr>`
	
	`// e.g.`
	`2 + 5`
	`(3 * X) + 1`
	
	`// you can combine it with an`
	`// asasignment, like so:`
	`NextTerm ← X + 1`
		`<expr> <operator> <expr>`
		
		`# e.g.`
		`2 + 5`
		`(3 * x) + 1`
		
		`# you can combine it with an`
		`# assignment, like so:`
		`next_term = x + 1`

Arithmetic Assignments
This is to perform a math operation on the variable itself, including incrementing a variable, etc.
	`// They DO NOT exist in pseudocode,`
	`// but may be substituted with:`
	
	`<ident> ← <ident> <operator> <expr>`
	
	`// e.g.`
	`Age ← Age + 1`
	`Temperature ← Temperature - 5`
		`<ident> <operator>= <expr>`
		
		`# e.g.`
		`age += 1`
		`temperature -= 5`

Comparison Operators
This is to check the relation between two values, such as equality, greater or less than, not equal to, etc.
	// Equality
	Age = 18
	
	// Greater than, less than
	Age > 18
	Age < 18
	
	// Greater than or equal to, less
	// than or equal to
	Age >= 18
	Age <= 18
	
	// Not equal to
	Age <> 18
		# Equality
		age == 18
		
		# Greater than, less than
		age > 18
		age < 18
		
		# Greater than or equal to, less
		# than or equal to
		age >= 18
		age <= 18
		
		# Not equal to
		age != 18

Boolean Expressions
This is akin to logic gates; it is to process one or two boolean values and evaluate it to True or False depending on the operator.
	// is one condition TRUE AND the
	// other one true?
	
	ConditionOne AND ConditionTwo
	
	// is one condition TRUE OR the
	// other one true?
	
	ConditionOne OR ConditionTwo
	
	// is the condition NOT true?
	
	NOT Condition
		# is one condition TRUE AND the
		# other one true?
		
		condition_one and condition_two
		
		# is one condition TRUE OR the
		# other one true?
		
		condition_one or condition_two
		
		# is the condition NOT true?
		
		not condition

Conditional Branching (if)
This is how to make a decision, a choice, to ask a question, whichever interpretation pleases you.
	// either:
	IF <condition>
	  THEN // PRESS SPACE TWICE!
	  <code> // PRESS SPACE TWICE!
	  ELSE
	  <code> // PRESS SPACE TWICE!
	ENDIF
	
	// or:
	IF <condition>
	  THEN
	  <code>
	ENDIF
	
	// e.g.
	IF Age > 18
	  THEN
	  OUTPUT "you can drink!"
	  ELSE
	  OUTPUT "you cannot drink..."
	ENDIF
		if <condition>:
		  <code> # PRESS SPACE 4 TIMES!
		else:
		  <code>
		
		# or
		if <condition>:
		  <code>
		
		# e.g.
		if age > 18:
		  print("you can drink!")
		else:
		  print("you cannot drink...")

<table>
<tr>
<td>Chained conditional branching (if-else if-else)
This is how to ask multiple questions in a row.

Note that in pseudocode, you should, i.e. THEN must be on a new line and indented by 2 spaces, and the code block by 4, ELSE by none, and the code block that follows by 2.

All other code blocks should be indented by 4 spaces.</td>
<td><pre>
// This does not exist in pseudocode, but can be emulated in the following way:

IF <condition>
  THEN
    <code>
  ELSE
    IF <condition>
      THEN
        <code>
      ELSE
        <code>
    ENDIF
ENDIF

// with the IF statement inside the
// larger ELSE statement being able
// to be repeated as many times as
// needed.

IF Age > 18
  THEN
    OUTPUT "You can drink!"
  ELSE
    IF Age > 16
      THEN
        OUTPUT "You can almost drink!"
      ELSE
        OUTPUT "You can't drink…"
    ENDIF
ENDIF
</pre></td>
<td><pre>
if <condition>:
    <code>
elif <condition>:
    <code>
else:
    <code>

e.g.
if age > 18:
    print("you can drink!")
elif age > 16:
    print("you can almost drink!")
else:
    print("you can't drink...")
</pre></td>
</tr>
<tr>
<td>Pattern Matching
This is like finding a value that matches the one that you have, and then doing something when you find it.
NOTE that using match in Python requires version 3.10 or later. If you use the latest version of beancode web or Thonny, you should be OK.</td>
<td><pre>
CASE OF <expr>
  <expr>: <statement>
  <expr>: <statement>
  ...
  // optionally,
  OTHERWISE <statement>
ENDCASE

// e.g.
CASE OF BottleMaterial
  "Plastic": OUTPUT "Unsustainable..."
  "Metal": OUTPUT "Sustainable!"
  "Glass": OUTPUT "Fragile..."
  "Paper": OUTPUT "WHY?"
  OTHERWISE OUTPUT "Unrecognized"
ENDCASE
</pre></td>
<td><pre>
match <expr>:
    case <expr>:
        <code>
    case <expr>:
        <code>
    ...
    # This is equivalent to OTHERWISE
    case _:
        <code>

match bottle_material:
    case "Plastic":
        print("Unsustainable...")
    case "Metal":
        print("Sustainable!")
    case "Glass":
        print("Fragile...")
    case "Paper":
        print("WHY?")
    case _:
        print("Unrecognized")
</pre></td>
</tr>
</table>

Pre-condition iteration (while)
 	
This is like repeatedly do tasks, while some condition is true (so to not infinitely loop).
	``
	WHILE <condition> DO
	    <code>
	ENDWHILE
	
	// e.g.
	WHILE Number > 1 DO
	    Number ← Number - 1
	    OUTPUT "The number is now", Number
	ENDWHILE
	``
		``
		while <condition>:
		    <code>
		
		# e.g.
		while number > 1:
		    number -= 1
		    print("The number is now",number)
		``

Post-condition iteration (repeat-until)
This is also used to repeatedly do tasks, while some condition is true, however the condition is checked after the code is run and not before.
 	
*In pseudocode, these post-condition loops have an inverted condition, meaning that it does something **until** the condition is true, **not while** it is true.*
	``
	REPEAT
	    <code>
	UNTIL <condition>
	
	// e.g.
	REPEAT
	    OUTPUT "Enter the password..."
	    INPUT Password
	    IF Password <> "Secret"
	      THEN
	        OUTPUT "Wrong..."
	    ENDIF
	UNTIL Password = "Secret"
	``
		``
		# Repeat-until loops do not exist in
		# Python due to it being mostly
		# redundant. You cannot do post-
		# condition loops either. You can
		# replicate the example like so:
		
		# negate the condition
		while password != "Secret":
		    password = input("Enter the
		password...")
		    if password != "Secret":
		        print("Wrong...")
		``

<table>
<tr>
<td><u>For Loops</u>
This is used to repeatedly do something until a counter reaches the end, which is specified.</td>
<td>

```
FOR <counter> ← <begin> TO <end>
    <code>
NEXT <counter>

FOR <counter> ← <begin> TO <end> STEP
<step>
    <code>
NEXT <counter>

// e.g.

FOR Counter ← 1 TO 5
    OUTPUT "There is a student called",
StudentNames[Counter], " in the class."
NEXT Counter

FOR OddNumber ← 1 TO 30 STEP 2
    OUTPUT OddNumber
NEXT OddNumber
```

</td>
<td>

```
for <counter> in range(<begin>,
<end>):
    <code>

for <counter> in range(<begin>,
<end>, <step>):
    <code>

# You don't have to write the first
# 0, but it is shown for clarity.
#
# The ending argument for range() is
# excluded from the numbers that it
# gives.
for counter in range(0, len(student_
names)):
    print("There is a student called
", student_names[counter], "in the
class.")

for odd_number in range(1, 30, 2):
    print(odd_number)
```

</td>
</tr>
<tr>
<td><u>Arrays</u>
This is used to store sequences of data, or grids/matrices of data.

Arrays in Pseudocode begin at 1, and they begin at 0 in Python.</td>
<td>

```
// In Pseudocode, arrays are STATIC,
// meaning that you cannot add or
// remove elements dynamically.
//
// Note that Pseudocode arrays can
// start at any index above 0 and end
// at any index above 0, denoted by l
// and h (meaning low and high).
//
// Declaring an ARRAY (1 dimensional)
//
// l is the lower bound, h is the
// higher bound
DECLARE <ident>:ARRAY[l:h] OF <type>

// Declaring an ARRAY (2 dimensional)
//
// l1 and h1 are the bounds of the
// outer array, l2 and h2 are the
// bounds of the inner array
DECLARE <ident>:ARRAY[l1:h1,l2:h2] OF
<type>

// e.g.
DECLARE StudentNames:ARRAY[1:5] OF
STRING

// Adapted from the IGCSE Syllabus
DECLARE TicTacToe:ARRAY[1:3,1:3] OF
CHAR
```

</td>
<td>

```
# Python does not have pseudocode
# ARRAYs, i.e. sequences of data of a
# fixed length, however, Python does
# have lists with push-back/pop-back
# functionality.
#
# It is impossible to trivially
# initialize arrays with N spaces for
# items already present.
#
# Note that Python lists must start
# at 0.
student_names: list[str]
student_names = [None for _ in
range(<h>)]

# Python does not have CHAR!
tic_tac_toe: list[list[str]]
tic_tac_toe = [[None for _ in
range(<h2>)] for _ in range(<h1>)]

# Assign to a list

student_names[1] = "Marcos"
tic_tac_toe[0][2] = "X"

# You can even assign a whole list!
student_names = ["Tom", "James",
"Jimmy", "John", "Peter"]
```

</td>
</tr>
</table>

	// Assign to an ARRAY (1 dimensional)
	StudentNames[2] ← "Marcos"
	TicTacToe[1,3] ← 'X'
	
	// Use an ARRAY
	<ident>[<index>] // 1D ARRAY
	<ident>[<index1>,<index2>] // 2D ARRAY
	
	// e.g.
	StudentNames[3] // get 3rd student name
	TicTacToe[2,1] // get the character at
	// y=2, x=1 on the Tic
	// Tac Toe board
		# Use a list
		<ident>[<index>] # 1D list
		<ident>[<index1>][<index2>] # 2D list
		
		# e.g.
		student_names[2] # get 3rd student
		# name
		tic_tac_toe[1][0] # get the character
		# at y=2, x=1 on
		# the Tic Tac Toe
		# board

<u>Procedures</u>
*These are repeatable sections of code that can be invoked (**called**) over and over as many times as needed. This might also be called a **subprogram,** or a **subroutine** (outdated).*
	// declaring procedures
	PROCEDURE <name>
	<code>
	ENDPROCEDURE
	
	PROCEDURE <name>(<parameter name>:
	<type>, <parameter name>:<type>, ...)
	<code>
	ENDPROCEDURE
	
	// e.g.
	PROCEDURE SayHello
	OUTPUT "Hello!"
	ENDPROCEDURE
	
	PROCEDURE Line(Size: **INTEGER**)
	FOR Length ← **1 TO** Size
	OUTPUT '-'
	NEXT Length
	ENDPROCEDURE
	
	// calling procedures
	CALL <name>
	CALL <name>(<parameter>,<parameter>...)
	
	// e.g.
	CALL SayHello
	CALL Line(10)
		# all "procedures" below are
		# technically functions, as Python
		# does not differentiate between
		# Procedures and Functions.
		
		# declaring procedures
		def <name>():
		<code>
		
		def <name>(<parameter name>: <type>,
		<parameter name>: <type>, ...):
		<code>
		
		# e.g.
		def say_hello():
		print("Hello!")
		
		def line(size: **int**):
		for length **in range**(1, size):
		print('-')
		
		# calling functions
		<name>()
		<name>(<parameter>, <parameter>...)
		
		# e.g.
		say_hello()
		line(10)

<u>Functions</u>
*These are repeatable sections of code, but they **return** values, meaning that they usually **process** or **give** data back to the site of invocation, also known as the*
	// declaring functions
	FUNCTION <name> **RETURNS** <type>
	<code>
	RETURN <expr> // you MUST return
	// something!
	ENDFUNCTION
	
	FUNCTION <name>(<parameter name>:
	<type>, <parameter name>: <type>, ...)
	RETURNS <type>
	<code>
	RETURN <expr> // you MUST return
	// something!
	ENDFUNCTION
		# declaring functions
		# **Note:** specifying return types with
		# **->** <type> is optional, and again,
		# done for clarity.
		def <name>() **->** <type>:
		<code>
		return <expr> # you MUST return
		# something!
		
		def <name>(<parameter name>: <type>,
		<parameter name>: <type>, ...) **->**
		<type>:
		<code>
		return <expr> # you MUST return
		# something!

caller.
 	
Procedures can also be referred to as ***fruitless*** *and Functions* ***fruitful*** *due to functions requiring a return value.*
 	
Python does not differentiate between functions and procedures.
	// e.g.
	FUNCTION GimmeFive RETURNS INTEGER
	    RETURN 5
	ENDFUNCTION
	
	FUNCTION AddOne(Num:INTEGER) RETURNS INTEGER
	    DECLARE Result:INTEGER
	    Result ← Num + 1
	    RETURN Result
	ENDFUNCTION
	
	// calling functions
	GimmeFive()
	AddOne(5)
	
	// ...or use them as expressions
	AddOne(GimmeFive())
	OUTPUT GimmeFive(), "+1 is", AddOne(5)
		# e.g.
		def gimme_five() -> int:
		    return 5
		
		def add_one(num: int) -> int:
		 result: int
		 result = num + 1
		 return result
		
		# calling functions
		gimme_five()
		add_one(5)
		
		# ...or use them as expressions
		add_one(gimme_five())
		print(gimme_five(), "+ 1 is",
		add_one(5))

<u>File I/O</u>
Self explanatory. This relates to writing data and reading data from files on the disk, hard drive, etc. that is ***not in memory****.*
	// file modes include READ and WRITE
	//
	// opening files
	OPENFILE <file name> FOR <file mode>
	
	// reading files (read into <variable>)
	READFILE <file name>, <variable>
	
	// writing files (write from <variable>)
	WRITEFILE <file name>, <variable>
	
	// closing files
	CLOSEFILE <file name>
	
	// e.g.
	OPENFILE "data.txt" FOR READ AND WRITE
	READFILE "data.txt", Content
	WRITEFILE "data.txt", Content + "Hi!"
	CLOSEFILE "data.txt"
		# READ corresponds to 'r'
		# WRITE corresponds to 'w'
		# READ AND WRITE corresponds to 'r+'
		# or 'w+'
		# opening files
		<ident> = open(<file name>, <file mode>)
		
		# reading files
		<variable> = <ident>.read()
		
		# writing files
		<ident>.write(<variable>)
		
		# closing files
		<ident>.close()
		
		# e.g.
		file = open("data.txt", "r+")
		content = file.read()
		file.write(content + "Hi!")
		file.close()

Appendix

Find the online copy here:

https://ezntek.com/posts/the-igcse-pseudocode-to-python-reference-guide-for-g1-and-g2-computer-science-20241018t2049/